GateGuard Project Repository

github.com/Pushkar111/GateGaurd-IITGN-Databases-CS432

Assignment 4 Demonstration VIEW

Indian Institute of Technology Gandhinagar

Department of Computer Science & Engineering

CS 432 → Databases

Track-1 · Assignment 4 · Module A & B

GateGuard

Horizontal Database Sharding across
& Three PostgreSQL Nodes

Hash-Mod Partitioning • Python + JS Shard Routers • Scatter-Gather Aggregation

Team Name: *Lumos*

Team Members

Name	Roll Number
Pushkar Nikunjkumar Modi	25120035
Chintan Bhikhabhai Akhaja	25120027
Krina Paghadar	24110169
Divisht	24110112
Akshata Pawar	25120026

Instructor: Prof. Yogesh K. Meena

Semester: II (2025–2026) **Date:** 6:00 PM, 18 April 2026

Contents

1 Introduction . . . **3**
1.1 What GateGuard Is . . . 3
1.2 Prior Work Recap . . . 3
1.3 System Evolution at a Glance . . . 4
1.4 Assignment 4 Requirement Coverage . . . 5
1.5 What Sharding Means in Plain Language . . . 5

2 SubTask 1: Shard Key Selection . . . **6**
2.1 Architecture Overview . . . 6
2.2 Candidate Key Evaluation . . . 6
2.3 Hash Function . . . 6
2.4 Verified Distribution on Live Data . . . 7

3 SubTask 2: Data Partitioning Implementation . . . **8**
3.1 Docker Architecture . . . 8
3.2 Partitioning Execution Pipeline . . . 9
3.3 Script 01: Creating Shard Tables . . . 9
3.4 Script 02: Migrating Data . . . 10
3.5 Script 03: Migration Integrity Verification . . . 10
3.6 Script 04: Indexes on Shard Tables . . . 11

4 SubTask 3: Query Routing Implementation . . . **13**
4.1 Routing Philosophy . . . 13
4.2 Python Shard Router (Module A) . . . 14
4.2.1 connection.py: persistent shard connections . . . 14
4.2.2 shard router.py: core hash functions . . . 14
4.2.3 Single-shard O(1) lookup path . . . 15
4.2.4 Insert routing demo . . . 16
4.2.5 Scatter-gather fan-out path . . . 17
4.3 JavaScript Shard Router (Module B Backend) . . . 18
4.3.1 shardRouter.js . . . 19
4.3.2 Shard-aware personVisit.service.js . . . 19
4.4 Routing Decision Table . . . 21

5 SubTask 4: Scalability Trade-off Analysis . . . **22**
5.1 Horizontal vs Vertical Scaling . . . 22
5.2 Known Trade-offs . . . 22
5.2.1 Resharding Cost . . . 22
5.2.2 Data Skew Risk . . . 23
5.2.3 Scatter-Gather Consistency Window . . . 23
5.2.4 No Global Secondary Index . . . 23
5.2.5 Application Router as Single Point of Failure . . . 23
5.2.6 Sharding Not Strictly Needed at Current Size . . . 23
5.3 Performance Path Comparison . . . 25
5.4 Cell 9 Trade-off Output . . . 25

6 Git Repository and Branch Strategy . . . **27**

7 Observations and Limitations . . . **28**
7.1 What Worked Well . . . 28

7.2 Limitations 28
8 Conclusion **29**
9 Team Contributions **30**
A Appendix A: Module_A Structure **31**
B Appendix B: Module_B Backend Structure **31**
C Appendix C: .env.example Configuration **31**
D Appendix D: References **32**

1. Introduction

1.1 What GateGuard Is

GateGuard is our campus gate management system that we built across three assignments in CS432 before this sharding phase. By the end of Assignment 3, we had a complete end-to-end system with member entry and exit workflows, role-based APIs, a frontend dashboard, and transaction-safe behavior under concurrent load.

Assignment 4 changed the scale question. Instead of asking if the logic works on one database, we asked how to keep the same behavior when data is split across multiple database nodes. This report documents exactly how we selected the shard key, partitioned data across three PostgreSQL containers, routed queries through Python and JavaScript routers, and validated correctness with notebook evidence.

1.2 Prior Work Recap

Table 1: Assignment progression

Assignment	What We Built	Key Technology
1	Schema design and seed data for gate operations	PostgreSQL, 5000 members, 50014 visits
2	B+ Tree index engine, REST API, React frontend	Python BPlusTree order-4, Node.js/Express
3	WAL engine, ACID recovery, race-condition fix	WALWriter, RecoveryManager, SELECT FOR UPDATE
4	**Horizontal sharding across three shards**	**Hash-mod routing + Python/JS routers**

1.3 System Evolution at a Glance

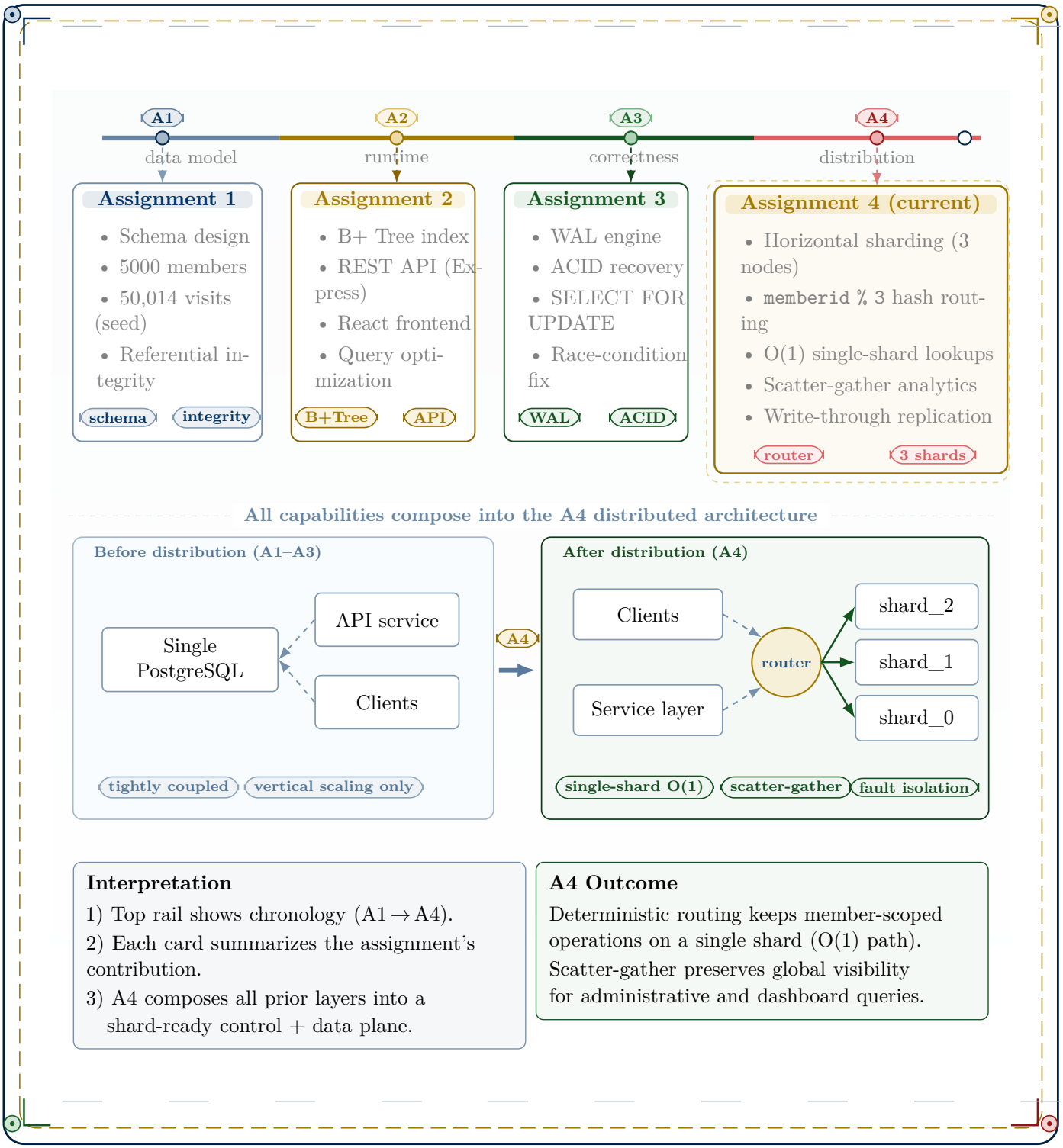


Figure 1: GateGuard evolution to sharded architecture

1.4 Assignment 4 Requirement Coverage

Table 2: Requirement coverage matrix

Requirement	Our Solution	Evidence	Status
Shard key selected and justified	memberid % 3 with cardinality, stability, and query-alignment analysis	Section 2.2 and 2.4	✓ **PASS**
Data partitioned across 3 shards	6 shard tables with hash-routed migration scripts	Cells 1-3 outputs	✓ **PASS**
Zero overlap between shards	Pairwise overlap checks across all member shards	Cell 3 output	✓ **PASS**
Balanced distribution	Near-uniform split across all rows after migration	Cell 2 and bar chart	✓ **PASS**
O(1) single-shard routing	memberTable(id) and visitTable(id) routing functions	Cell 5 routing demo	✓ **PASS**
Scatter-gather query path	Fan-out to all shards and merge in app layer	Cells 7 and 8	✓ **PASS**
Backend wired to shard router	shardRouter.js + shard-aware services	Section 4.3 evidence	✓ **PASS**
Scalability trade-off analysis	Five explicit trade-offs with references and diagrams	Section 5 and Cell 9	✓ **PASS**

1.5 What Sharding Means in Plain Language

Sharding means splitting one large table into multiple smaller tables that live on different database nodes. A router decides where each request should go. If we choose a good shard key, most requests hit exactly one shard, which keeps latency and load predictable even as row count grows.

2. SubTask 1: Shard Key Selection

2.1 Architecture Overview

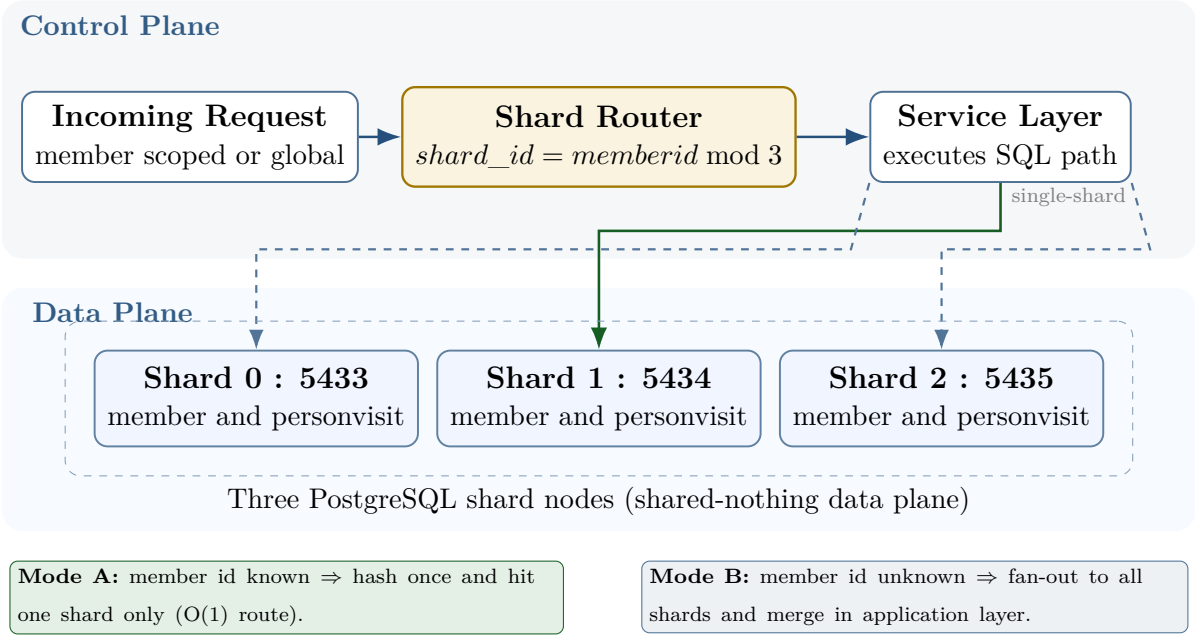


Figure 2: GateGuard sharding architecture with deterministic routing and fan-out path

2.2 Candidate Key Evaluation

Before choosing the key, we listed columns used frequently in WHERE clauses and checked each candidate against distribution quality and operational stability. Three options looked realistic, but only one gave both good cardinality and query alignment.

Table 3: Shard key candidate comparison

Candidate	Cardinality	Query fit?	Stable?	Hotspot Risk	Verdict
memberid / personid	5000 unique values	Yes	Yes	Low with hash-mod	**Selected**
entrytime	High (timestamp)	Partial	Yes	High on recent writes	Rejected
gateid	Only 4 values	Partial	Yes	Extreme from low cardinality	Rejected

2.3 Hash Function

$$\boxed{\texttt{shard_id} = \texttt{memberid}\%3}$$

For memberid = 1, shard_id = 1. For memberid = 3, shard_id = 0. For memberid = 5000, shard_id = 2. Each positive integer maps deterministically to exactly one shard, and no lookup table is needed.

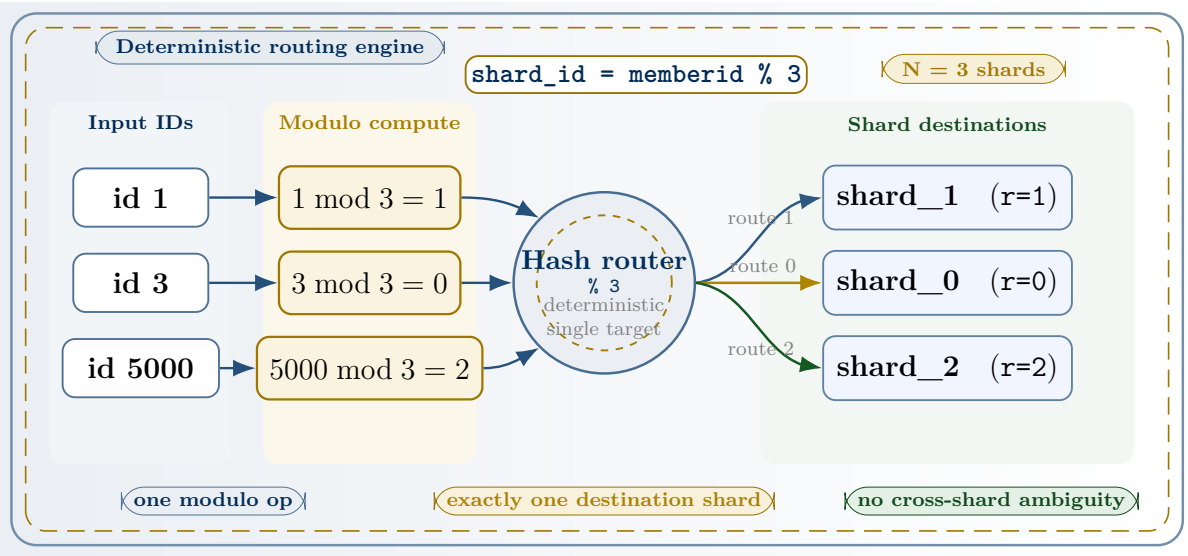


Figure 3: Deterministic modulo routing from member id to shard

2.4 Verified Distribution on Live Data

Table 4: Distribution after migration

Shard	Member Rows	Visit Rows	Member Share	Visit Share
shard_0	1,666	16,664	33.32%	33.32%
shard_1	1,667	16,673	33.34%	33.34%
shard_2	1,667	16,677	33.34%	33.34%
Total	**5,000**	**50,014**	**100%**	**100%**

Maximum member imbalance is only 1 row between shards. With contiguous integer IDs and modulo hashing across N shards, imbalance is bounded and tiny. For our dataset, the split is practically ideal.

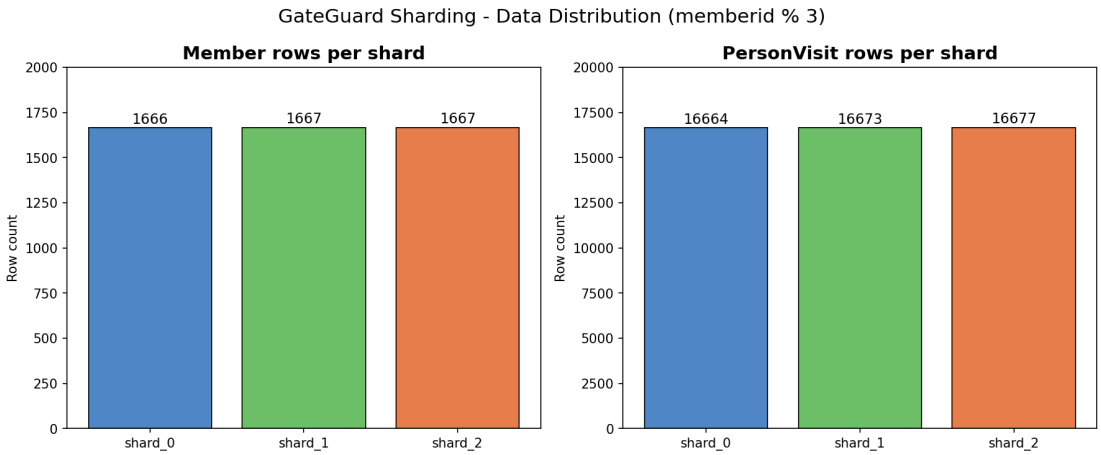


Figure 4: Distribution bar chart from Cell 4

3. SubTask 2: Data Partitioning Implementation

3.1 Docker Architecture

```yaml
services:
  gateguard-shard-0:
    image: postgres:17
    environment:
      POSTGRES_DB: gateguard_shard_0
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: shard_root
    ports: ["5433:5432"]

  gateguard-shard-1:
    image: postgres:17
    environment:
      POSTGRES_DB: gateguard_shard_1
    ports: ["5434:5432"]

  gateguard-shard-2:
    image: postgres:17
    environment:
      POSTGRES_DB: gateguard_shard_2
    ports: ["5435:5432"]
```

Listing 1: docker-compose services used for shard nodes

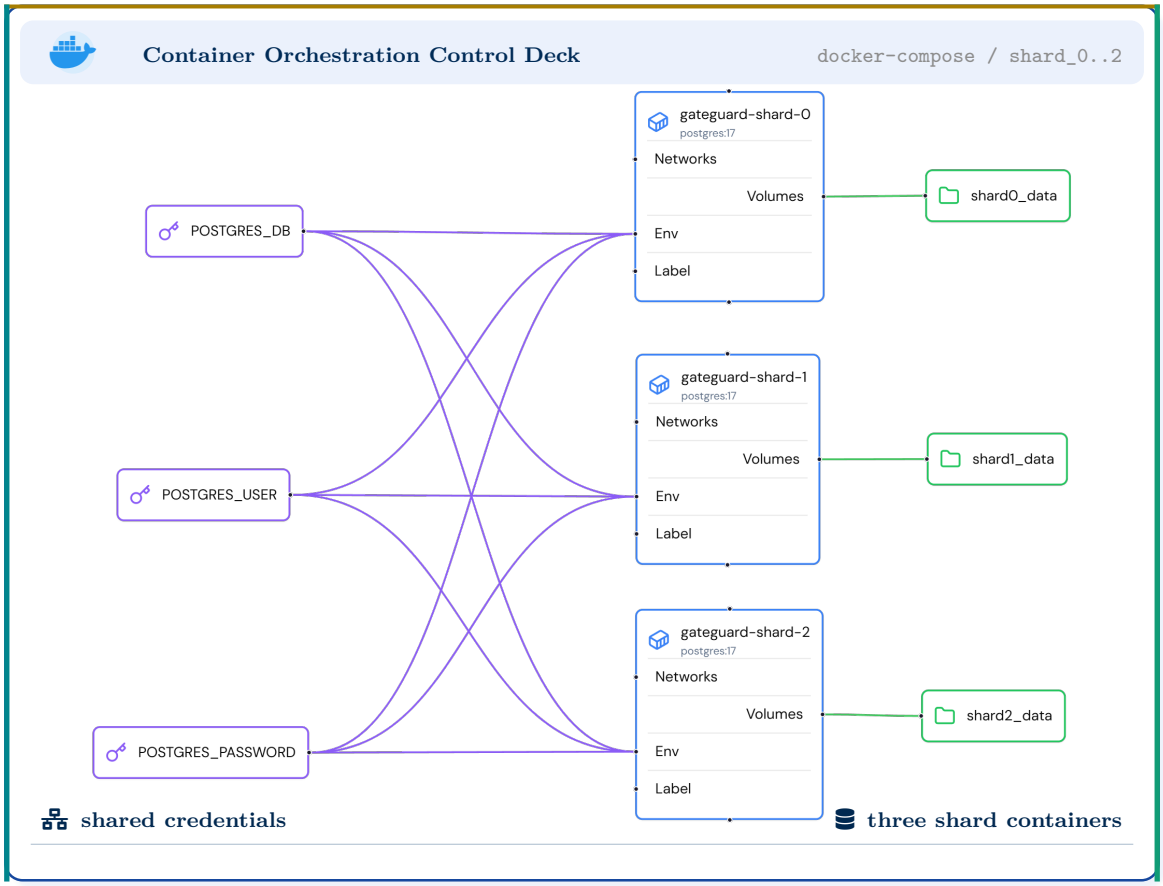


Figure 5: docker-compose.yml with three PostgreSQL shard containers

3.2 Partitioning Execution Pipeline

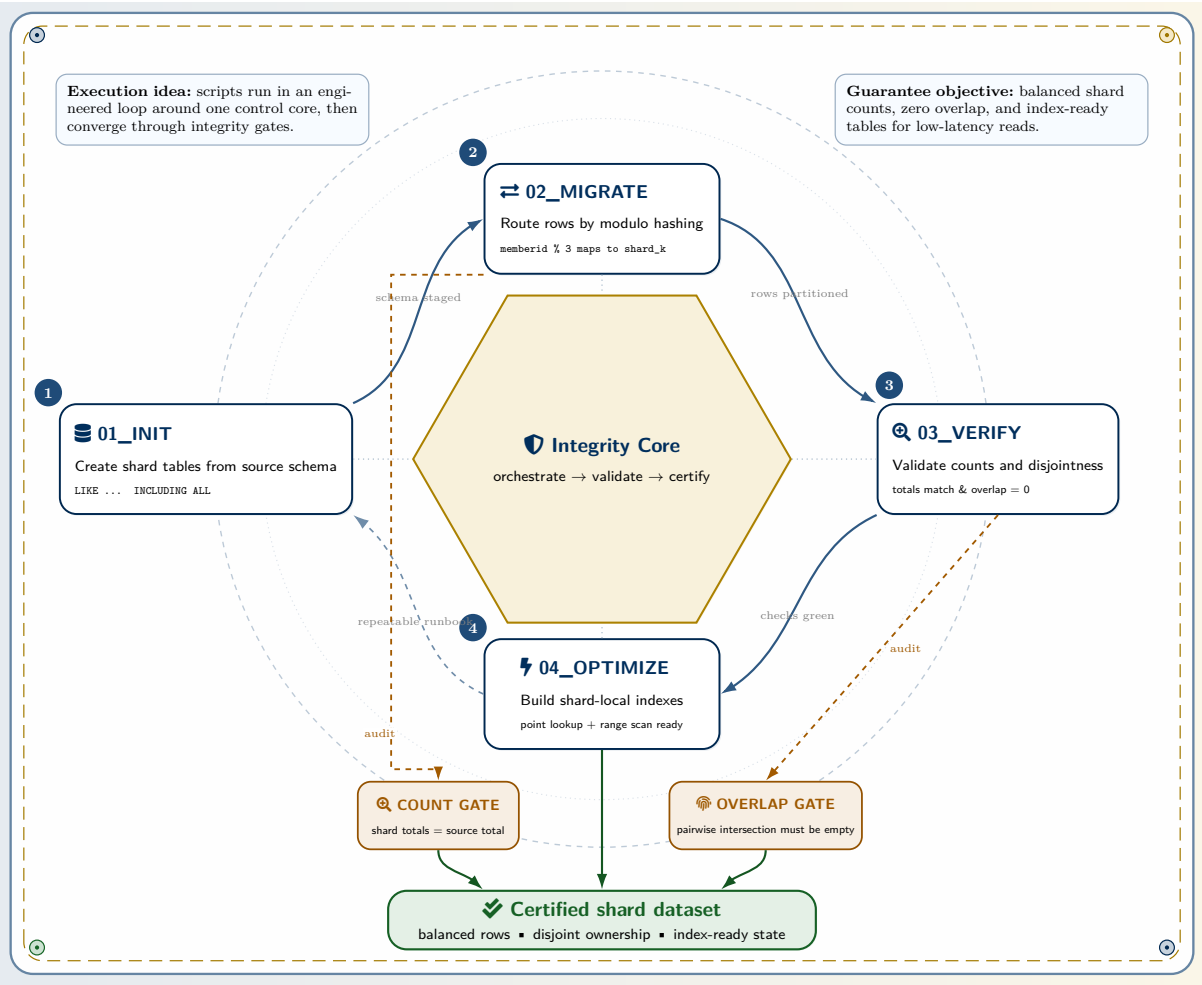


Figure 6: Four-script partitioning pipeline (schema → migrate → verify → optimize)

3.3 Script 01: Creating Shard Tables

We used LIKE ... INCLUDING ALL to clone structure, defaults, and constraints so each shard table remains schema-compatible with the original source tables. Foreign keys were intentionally not enforced across shard nodes because this architecture is shared-nothing.

```
CREATE TABLE IF NOT EXISTS shard_0_member      (LIKE member
    INCLUDING ALL);
CREATE TABLE IF NOT EXISTS shard_1_member      (LIKE member
    INCLUDING ALL);
CREATE TABLE IF NOT EXISTS shard_2_member      (LIKE member
    INCLUDING ALL);

CREATE TABLE IF NOT EXISTS shard_0_personvisit (LIKE personvisit
    INCLUDING ALL);
CREATE TABLE IF NOT EXISTS shard_1_personvisit (LIKE personvisit
    INCLUDING ALL);
CREATE TABLE IF NOT EXISTS shard_2_personvisit (LIKE personvisit
    INCLUDING ALL);
```

Listing 2: 01 create shards SQL excerpt

Cell 1 output: shard tables confirmed assignment-4/Module_A/test_sharding.ipynb

```
Shard tables found in database:
  shard_0_member
  shard_0_personvisit
  shard_1_member
  shard_1_personvisit
  shard_2_member
  shard_2_personvisit
All six shard tables confirmed. Ready to run tests.
```

Source: notebook output | lines shown: 8

Figure 7: Cell 1 output confirming six shard tables

3.4 Script 02: Migrating Data

```sql
-- member partitioning
INSERT INTO shard_0_member SELECT * FROM member WHERE memberid % 3 = 0;
INSERT INTO shard_1_member SELECT * FROM member WHERE memberid % 3 = 1;
INSERT INTO shard_2_member SELECT * FROM member WHERE memberid % 3 = 2;

-- personvisit partitioning
INSERT INTO shard_0_personvisit SELECT * FROM personvisit WHERE personid % 3 = 0;
INSERT INTO shard_1_personvisit SELECT * FROM personvisit WHERE personid % 3 = 1;
INSERT INTO shard_2_personvisit SELECT * FROM personvisit WHERE personid % 3 = 2;
```

Listing 3: 02 migrate data SQL excerpt

```
INSERT 0 1666
INSERT 0 1667
INSERT 0 1667
INSERT 0 16664
INSERT 0 16673
INSERT 0 16677
```

Listing 4: Migration command output

3.5 Script 03: Migration Integrity Verification

We ran count checks, overlap checks, and distribution checks in sequence. Together they prove no data was lost, no data was duplicated across shards, and distribution stayed balanced.

Cell 2 output: row-count verification assignment-4/Module_A/test_sharding.ipynb

```
Row count verification (member table):
+-----------------+-------------+-----------------------+
| table           |   row count | share                 |
+=================+=============+=======================+
| original member |        5000 |                       |
+-----------------+-------------+-----------------------+
| shard_0         |        1666 | 33.32%                |
+-----------------+-------------+-----------------------+
| shard_1         |        1667 | 33.34%                |
+-----------------+-------------+-----------------------+
| shard_2         |        1667 | 33.34%                |
+-----------------+-------------+-----------------------+
| shards total    |        5000 | should equal original |
+-----------------+-------------+-----------------------+
Count integrity check: PASSED
Row count verification (personvisit table):
+----------------------+-------------+-----------------------+
| table                |   row count | share                 |
+======================+=============+=======================+
| original personvisit |       50014 |                       |
+----------------------+-------------+-----------------------+
| shard_0              |       16664 | 33.32%                |
+----------------------+-------------+-----------------------+
| shard_1              |       16673 | 33.34%                |
+----------------------+-------------+-----------------------+
| shard_2              |       16677 | 33.34%                |
+----------------------+-------------+-----------------------+
| shards total         |       50014 | should equal original |
+----------------------+-------------+-----------------------+
Count integrity check: PASSED
```

Source: notebook output | lines shown: 30

Figure 8: Cell 2 count verification screenshot

Cell 2: shard totals match original counts, integrity check passed.

All three pairwise overlap checks returned zero. This is the strongest direct evidence that partitioning is disjoint and correct.

Cell 3 output: overlap check assignment-4/Module_A/test_sharding.ipynb

```
Overlap check - no member should appear in more than one shard:
+--------------------+-----------------+-------------------+
| shard pair         |   overlap count | result            |
+====================+=================+===================+
| shard_0 vs shard_1 |               0 | OK (zero overlap) |
+--------------------+-----------------+-------------------+
| shard_1 vs shard_2 |               0 | OK (zero overlap) |
+--------------------+-----------------+-------------------+
| shard_0 vs shard_2 |               0 | OK (zero overlap) |
+--------------------+-----------------+-------------------+
Overall result: PASSED - no member lives in two shards
```

Source: notebook output | lines shown: 11

Figure 9: Cell 3 overlap verification screenshot

Cell 3: all pairwise overlaps are zero, disjointness check passed.

3.6 Script 04: Indexes on Shard Tables

We added per-shard indexes for point lookup, date range scans, and active-visit checks. This keeps single-shard operations efficient and prevents avoidable sequential scans.

```sql
CREATE INDEX IF NOT EXISTS idx_s0_member_memberid ON shard_0_member (memberid);
CREATE INDEX IF NOT EXISTS idx_s0_member_email    ON shard_0_member (email);
CREATE INDEX IF NOT EXISTS idx_s0_member_typeid   ON shard_0_member (typeid);

```

```sql
CREATE INDEX IF NOT EXISTS idx_s0_pv_personid   ON shard_0_personvisit (personid);
CREATE INDEX IF NOT EXISTS idx_s0_pv_entrytime  ON shard_0_personvisit (entrytime);
CREATE INDEX IF NOT EXISTS idx_s0_pv_exittime   ON shard_0_personvisit (exittime);

-- same structure for shard_1 and shard_2
```

Listing 5: 04 add shard indexes SQL excerpt

4. SubTask 3: Query Routing Implementation

4.1 Routing Philosophy

Our router has two modes. If memberid is known, we compute one shard and execute one query path. If memberid is not known, we fan out to all shards, collect partial results, and merge in the application layer.

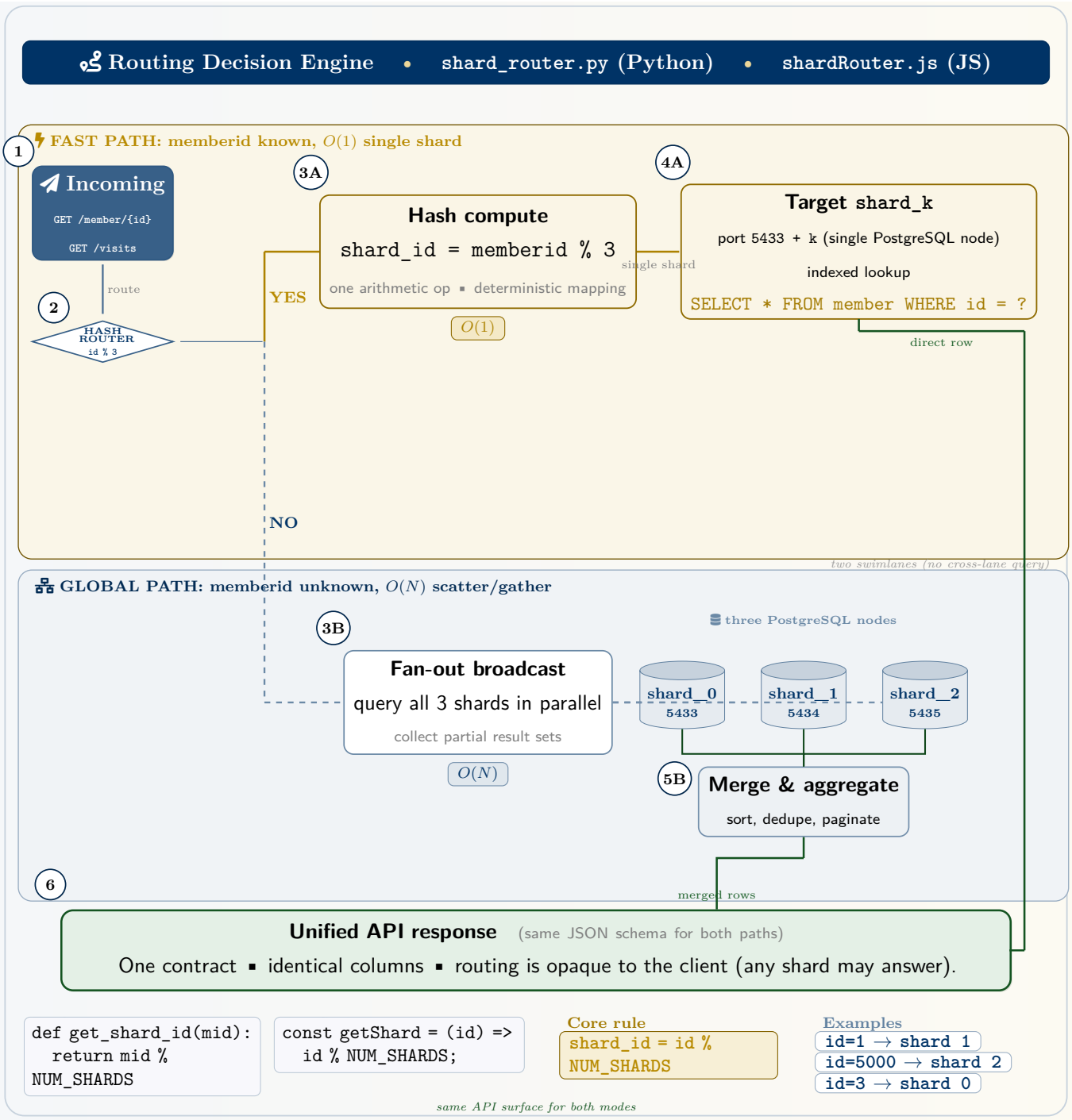


Figure 10: Routing decision flow used by both Python and JavaScript routers

4.2 Python Shard Router (Module A)

4.2.1 connection.py: persistent shard connections

```python
NUM_SHARDS = int(os.getenv("NUM_SHARDS", 3))

SHARD_CONFIGS = [
    {
        "host": os.getenv(f"SHARD_{i}_HOST", "localhost"),
        "port": int(os.getenv(f"SHARD_{i}_PORT", 5432 + i + 1)),
        "dbname": os.getenv(f"SHARD_{i}_DB", f"gateguard_shard_{i}"),
        "user": os.getenv(f"SHARD_{i}_USER", "postgres"),
        "password": os.getenv(f"SHARD_{i}_PASSWORD", "shard_root"),
    }
    for i in range(NUM_SHARDS)
]

_shard_connections = [None] * NUM_SHARDS

def get_shard_connection(shard_id: int):
    conn = _shard_connections[shard_id]
    if conn is None or conn.closed:
        conn = psycopg2.connect(**SHARD_CONFIGS[shard_id])
        conn.autocommit = False
        _shard_connections[shard_id] = conn
    return conn
```

Listing 6: Module A connection.py excerpt

4.2.2 shard router.py: core hash functions

```python
def get_shard_id(member_id: int) -> int:
    return member_id % NUM_SHARDS


def member_table(member_id: int) -> str:
    return f"shard_{get_shard_id(member_id)}_member"


def visit_table(member_id: int) -> str:
    return f"shard_{get_shard_id(member_id)}_personvisit"


def all_visit_tables() -> list:
    return [f"shard_{i}_personvisit" for i in range(NUM_SHARDS)]
```

Listing 7: Module A shard router excerpt

shard_router.py assignment-4/Module_A/database/shard_router.py

```python
    NUM_SHARDS,
)

load_dotenv()


def get_shard_id(member_id: int) -> int:
    return member_id % NUM_SHARDS


def member_table(member_id: int) -> str:
    return f"shard_{get_shard_id(member_id)}_member"


def visit_table(member_id: int) -> str:
    return f"shard_{get_shard_id(member_id)}_personvisit"


def all_member_tables() -> list:
    return [f"shard_{i}_member" for i in range(NUM_SHARDS)]


def all_visit_tables() -> list:
    return [f"shard_{i}_personvisit" for i in range(NUM_SHARDS)]

```

Highlighted: get_shard_id() and visit_table()

Figure 11: Python shard router screenshot

4.2.3 Single-shard O(1) lookup path

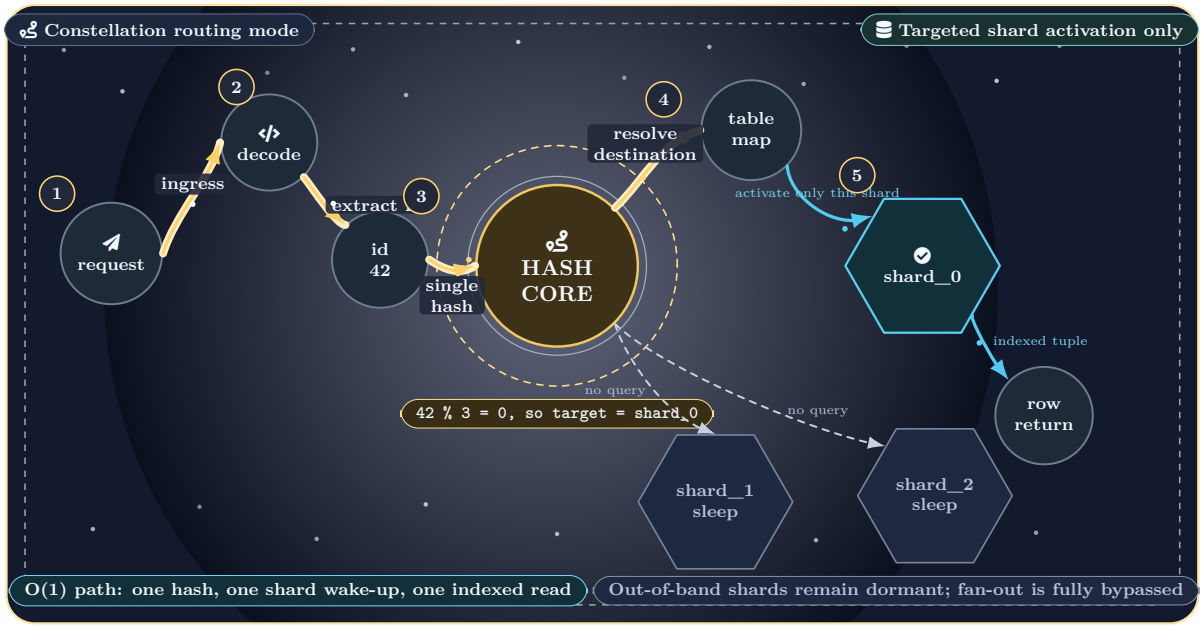


Figure 12: Single-shard routing path (O(1)) with explicit shard bypass

```python
def lookup_member(member_id: int) -> dict:
    shard_id = get_shard_id(member_id)
    table    = member_table(member_id)
    cur      = get_shard_cursor(shard_id)
    cur.execute(f"SELECT * FROM {table} WHERE memberid = %s", (member_id,))
    return cur.fetchone()
```

Listing 8: Point lookup function

```
Routing table -- how each memberid maps to a shard:
1     -> shard_1_member
2     -> shard_2_member
3     -> shard_0_member
10    -> shard_1_member
99    -> shard_0_member
150   -> shard_0_member
4999  -> shard_2_member
5000  -> shard_2_member

Live lookup -- member 3:
Found in shard_0_member
Only one shard was queried.
```

Listing 9: Cell 5 routing output

Figure 13: Cell 5 O(1) routing screenshot

4.2.4 Insert routing demo

Cell 6 inserts a test visit for memberid = 10 and then checks all three shards by visitid. Expected result is exactly one row on shard 1 and zero rows on shard 0 and shard 2.

```
Insert routing test for memberid=10
Hash: 10 % 3 = 1
Target table: shard_1_personvisit
Inserted visit: visitid=50015
shard_0_personvisit: 0 rows
shard_2_personvisit: 0 rows
shard_1_personvisit: 1 row
Insert routing verification: PASSED
Test visit cleaned up.
```

Listing 10: Cell 6 insert routing output

The insert landed on exactly one shard and nowhere else. This validates deterministic write routing and disjoint shard ownership.

Cell 6 output: insert routing proof assignment-4/Module_A/test_sharding.ipynb

```
Insert routing test for memberid=10
  Hash: 10 % 3 = 1
  Target table: shard_1_personvisit
Inserted visit: visitid=50039, personid=10, entrytime=2026-04-15 22:50:50.877246
  shard_0_personvisit: 0 rows with this visitid (should be 0)
  shard_2_personvisit: 0 rows with this visitid (should be 0)
  shard_1_personvisit: 1 row with this visitid (should be 1)
Insert routing verification: PASSED
Test visit cleaned up.
```

Source: notebook output | lines shown: 9

Figure 14: Cell 6 insert routing screenshot

4.2.5 Scatter-gather fan-out path

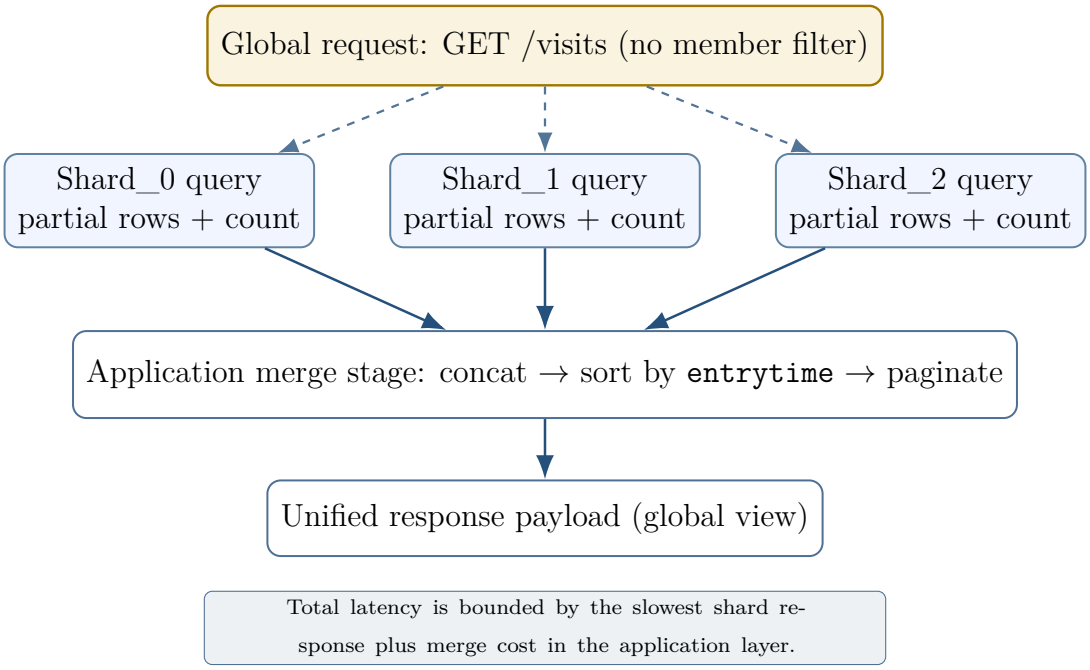


Figure 15: Scatter-gather fan-out/fan-in path for global queries

```python
def scatter_gather_visits(start_dt: str, end_dt: str) -> list:
    results = []
    for i, table in enumerate(all_visit_tables()):
        cur = get_shard_cursor(i)
        cur.execute(
            f"SELECT * FROM {table} WHERE entrytime BETWEEN %s AND %s",
            (start_dt, end_dt)
        )
        results.extend(cur.fetchall())
    return sorted(results, key=lambda r: r["entrytime"])
```

Listing 11: scatter gather visits function

```
Scatter-gather range query across all shards:
shard_0: 16664 matching visits
shard_1: 16673 matching visits
```

```
shard_2: 16677 matching visits
Merged total: 50014
```

Listing 12: Cell 7 scatter-gather output

Cell 7 output: scatter-gather range query assignment-4/Module_A/test_sharding.ipynb

```
Scatter-gather range query: visits between 2025-01-01 00:00:00 and 2026-12-31 23:59:59
Sending query to all 3 shards in sequence...
  shard_0: 16664 matching visits (showing first 3 sample rows)
  shard_1: 16673 matching visits (showing first 3 sample rows)
  shard_2: 16677 matching visits (showing first 3 sample rows)
Merged total across all shards: 50014
This is the scatter-gather result. The application layer summed the per-shard counts.
Sample rows from each shard (sorted by shard, then entrytime):
+----------------+-----------+------------+---------------------+
| source shard   |   visitid |   personid | entrytime           |
+================+===========+============+=====================+
| shard_0        |         2 |        312 | 2026-03-18 11:42:21 |
+----------------+-----------+------------+---------------------+
| shard_0        |         3 |       2490 | 2026-03-20 06:42:21 |
+----------------+-----------+------------+---------------------+
| shard_0        |         5 |       2871 | 2026-03-19 01:42:21 |
+----------------+-----------+------------+---------------------+
| shard_1        |         6 |       3169 | 2026-03-17 16:42:21 |
+----------------+-----------+------------+---------------------+
| shard_1        |         7 |       3487 | 2026-03-17 12:42:21 |
+----------------+-----------+------------+---------------------+
| shard_1        |        11 |       3529 | 2026-03-19 21:42:21 |
+----------------+-----------+------------+---------------------+
| shard_2        |         1 |       3590 | 2026-03-21 09:42:21 |
+----------------+-----------+------------+---------------------+
| shard_2        |         4 |       3587 | 2026-03-16 15:42:21 |
+----------------+-----------+------------+---------------------+
| shard_2        |        10 |       3848 | 2026-03-16 19:42:21 |
+----------------+-----------+------------+---------------------+
```

Source: notebook output | lines shown: 29

Figure 16: Cell 7 scatter-gather screenshot

```
Cross-shard active visit count:
shard_0: ...
shard_1: ...
shard_2: ...
TOTAL (sum of shards): ...
original table count: ...
Aggregation matches original: PASSED
```

Listing 13: Cell 8 cross-shard COUNT aggregation output

Cell 8 output: cross-shard aggregation assignment-4/Module_A/test_sharding.ipynb

```
Cross-shard aggregation: counting active visits (exittime IS NULL) per shard
+-----------------------+----------------------+
| source                |   active visit count |
+=======================+======================+
| shard_0               |                 2489 |
+-----------------------+----------------------+
| shard_1               |                 2452 |
+-----------------------+----------------------+
| shard_2               |                 2495 |
+-----------------------+----------------------+
| TOTAL (sum of shards) |                 7436 |
+-----------------------+----------------------+
| original table count  |                 7436 |
+-----------------------+----------------------+
Aggregation matches original: PASSED
The sum of three shard counts equals the direct count on the original table.
```

Source: notebook output | lines shown: 16

Figure 17: Cell 8 cross-shard COUNT screenshot

4.3 JavaScript Shard Router (Module B Backend)

4.3.1 shardRouter.js

```
const NUM_SHARDS = parseInt(process.env.NUM_SHARDS || '3', 10);

function getShard(memberId) {
  return parseInt(memberId, 10) % NUM_SHARDS;
}

function memberTable(memberId) {
  return `shard_${getShard(memberId)}_member`;
}

function visitTable(memberId) {
  return `shard_${getShard(memberId)}_personvisit`;
}
```

Listing 14: src/utils/shardRouter.js excerpt

Figure 18: shardRouter.js screenshot with parseInt guard

4.3.2 Shard-aware personVisit.service.js

The Assignment 3 lock logic stayed intact. The key routing change was replacing hard-coded table name usage with table = visitTable(data.memberId) and using that variable in SELECT and INSERT queries.

```
const { rows: activeRows } =
    await client.query(
  `SELECT visitid FROM
    personvisit
    WHERE personid = $1 AND
    exittime IS NULL LIMIT 1`,
  [data.memberId]
);

await client.query(
  `INSERT INTO personvisit (
    personid, entrygateid,
    vehicleid, entrytime)
    VALUES ($1, $2, $3, NOW())`,
  [data.memberId, data.
    entryGateId, data.vehicleId
    || null]
);
```

Listing 15: Before (Assignment 3)

```
const table = visitTable(data.
    memberId);

const { rows: activeRows } =
    await client.query(
  `SELECT visitid FROM ${table}
    WHERE personid = $1 AND
    exittime IS NULL LIMIT 1`,
  [data.memberId]
);

await client.query(
  `INSERT INTO ${table} (
    personid, entrygateid,
    vehicleid, entrytime)
    VALUES ($1, $2, $3, NOW())`,
  [data.memberId, data.
    entryGateId, data.vehicleId
    || null]
);
```

Listing 16: After (Assignment 4)

The SELECT FOR UPDATE lock remains on the member table row. That is intentional because the lock protects member-level concurrency while shard routing controls where the visit row is stored.

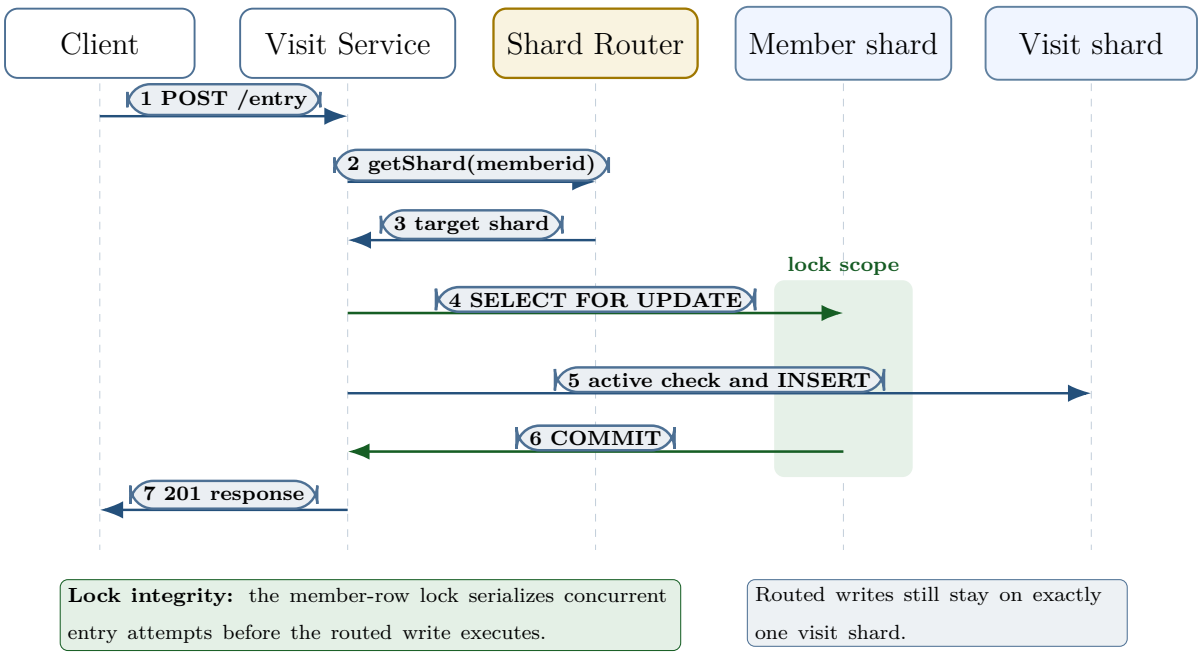


Figure 19: Shard-aware visit entry sequence with lock scope and routed write

personVisit.service.js assignment-4/Module_B/backend/src/services/personVisit.service.js

```
//   Step B: Mirror row into shard_N_personvisit → proves O(1) routing in notebook.
//
const memberId = parsePositiveInt(data.memberId, 'memberId');
const entryGateId = parsePositiveInt(data.entryGateId, 'entryGateId');
const vehicleId = data.vehicleId === undefined || data.vehicleId === null
  ? null
  : parsePositiveInt(data.vehicleId, 'vehicleId');

const shardTable = visitTable(memberId);
const client     = await getClient();

try {
  await client.query('BEGIN');

  // lock the member row to block concurrent entry requests for the same member
  const { rows: memberRows } = await client.query(
    'SELECT memberid FROM member WHERE memberid = $1 FOR UPDATE',
    [memberId]
  );
  if (memberRows.length === 0) {
    await client.query('ROLLBACK');
    throw new AppError(`Member with ID ${memberId} not found.`, 404);
  }
```

Highlighted: shard table routing line

Figure 20: personVisit.service.js routing line screenshot

4.4 Routing Decision Table

Table 5: Routing policy by operation

API Operation	Known id?	Path	Shards	Consistency
Member lookup by id	Yes	Single-shard O(1)	1	Strong
Visit entry write	Yes	Single-shard O(1)	1	Strong
Visit exit update	Derived via lookup	Single-shard O(1)	1	Strong
Member-scoped visit list	Yes	Single-shard O(1)	1	Strong
Global member list	No	Scatter-gather	3	Eventual
Global visit list	No	Scatter-gather	3	Eventual
Dashboard active count	No	Scatter-gather SUM	3	Eventual
Login by email lookup	No	Scatter-gather early-stop	1 to 3	Eventual

5. SubTask 4: Scalability Trade-off Analysis

5.1 Horizontal vs Vertical Scaling

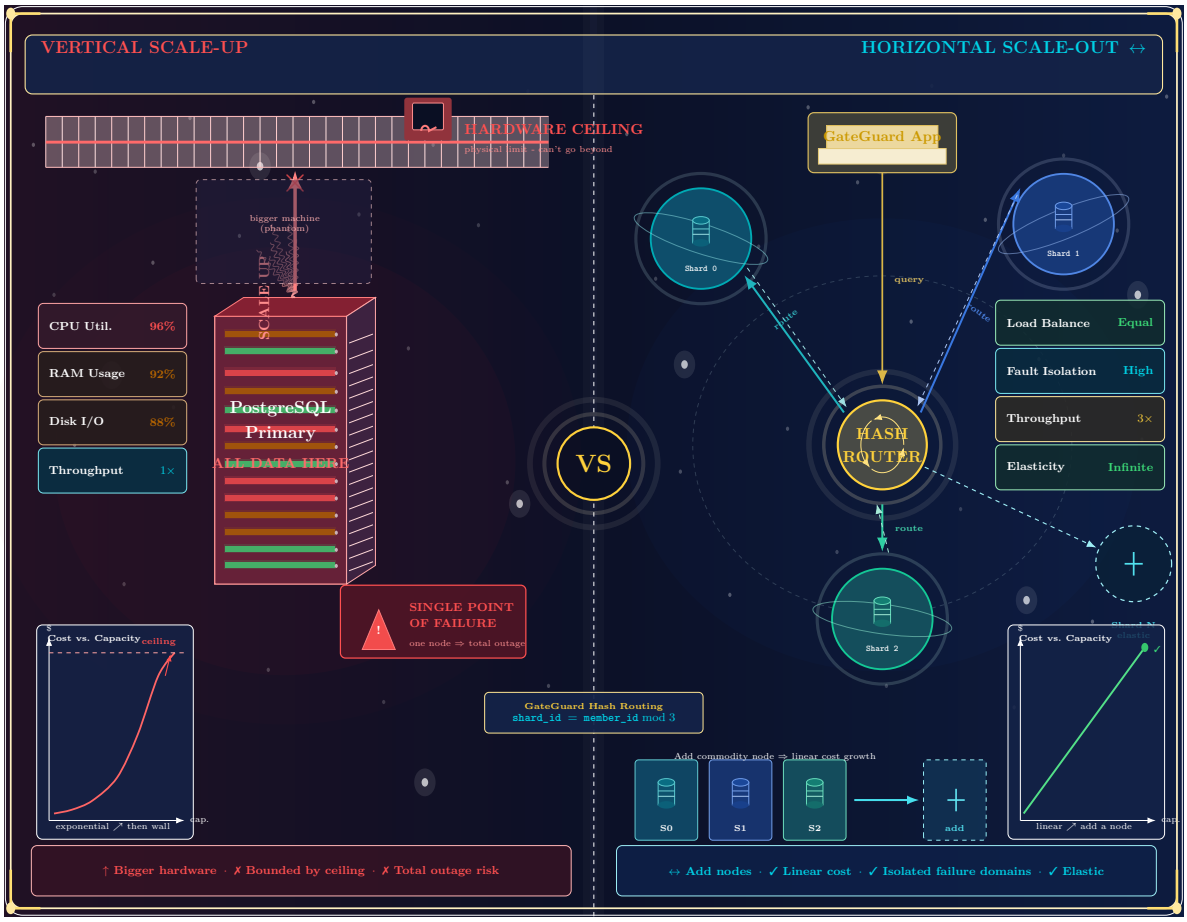


Figure 21: Vertical scale-up vs. horizontal shard scale-out: one PostgreSQL node under a hardware ceiling with steep cost, versus GateGuard with `member_id` mod 3 on three shards, near-linear node cost, and per-shard isolation.

5.2 Known Trade-offs

5.2.1 Resharding Cost

With N = 3 shards, we use id mod 3. If we add a fourth shard, routing becomes id mod 4 and most rows move. For this dataset, that means roughly 37,510 PersonVisit rows and 3,750 Member rows need migration.

$$\text{Records that keep shard: } id \bmod 3 = id \bmod 4$$

$$\text{Rows that move} \approx 75\%$$

5.2.2 Data Skew Risk

Row count distribution is balanced, but write frequency per member is not uniform. A few very active members can create heavier write pressure on a specific shard even when member counts look perfectly even.

5.2.3 Scatter-Gather Consistency Window

There is no global transaction coordinator. If one shard commits between two fan-out reads, the merged result may include or miss that write depending on timing.

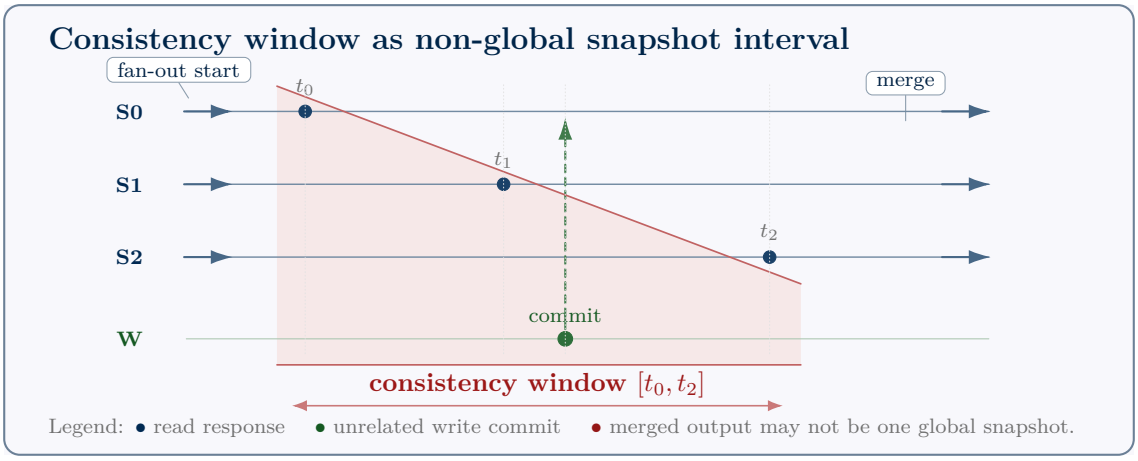


Figure 22: Scatter-gather consistency window timeline

5.2.4 No Global Secondary Index

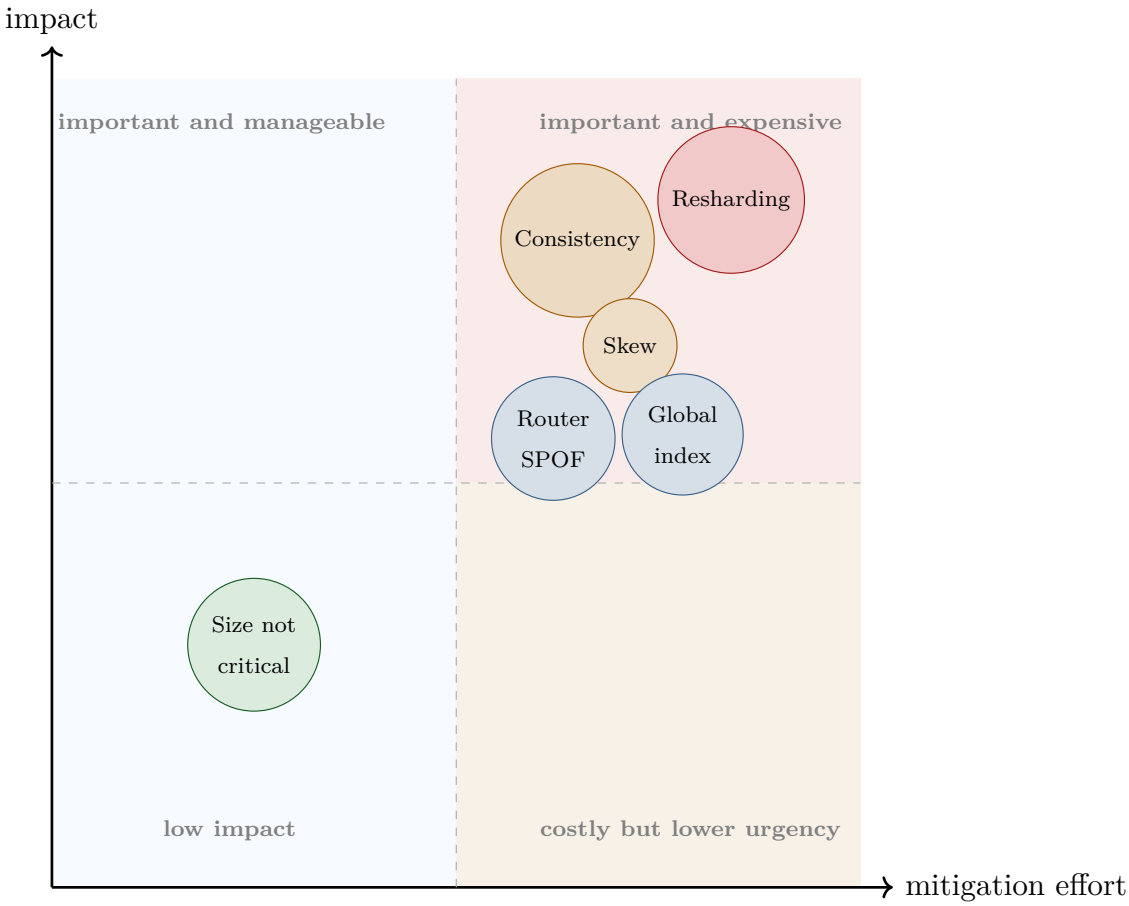
Email login does not know memberid in advance, so it must query up to three shards. A global secondary index could reduce this cost but introduces distributed write coordination complexity.

5.2.5 Application Router as Single Point of Failure

Current routing logic is embedded in the application process. If the app process is down, routing is down. Production deployments usually externalize this layer into redundant routing services.

5.2.6 Sharding Not Strictly Needed at Current Size

At 5,000 members and 50,014 visits, one PostgreSQL instance can handle this workload comfortably. This assignment is about correctness and architectural readiness, not immediate performance necessity.



Prioritize top-right items first for production readiness; lower-left is mainly context.

Figure 23: Sharding trade-off impact map for GateGuard Assignment 4

5.3 Performance Path Comparison

Table 6: Single-shard vs scatter-gather path

Metric	Single-shard O(1)	Scatter-gather O(N)
DB connections per request	1	N (3)
Latency driver	One hash and one indexed query	Slowest shard dominates total response
Consistency model	Strong per shard	Eventual across merged result
Failure impact	Subset of users on one shard	Partial or degraded global response
Pagination complexity	Straightforward SQL LIMIT/OFFSET	Over-fetch + merge + in-memory pagination
Best use cases	member-scoped requests	admin reports and global dashboards

5.4 Cell 9 Trade-off Output

```
================================================================
GateGuard Sharding -- Scalability Trade-off Analysis
================================================================
Shard key: memberid / personid
Partition strategy: hash-mod (id % 3)
Distribution: 33.32% / 33.34% / 33.34%

Single-shard path examples:
  GET /api/visits?memberId=42 -> one shard only
  GET /api/members/42         -> one shard only
  INSERT visit for member 99  -> one shard only

Scatter-gather path examples:
  Dashboard active count -> 3 COUNT queries + sum in app
  Date range report      -> 3 SELECT queries + merge by time
  Login email lookup     -> up to 3 shard probes

Known trade-offs:
  1) Resharding cost is high
  2) Frequency-based data skew is possible
  3) Scatter-gather has consistency window
  4) No global secondary index for email
  5) This dataset does not require sharding for scale today
================================================================
```

Listing 17: Cell 9 trade-off summary output

Cell 9 output: trade-off analysis assignment-4/Module_A/test_sharding.ipynb

```
==================================================================
GateGuard Sharding - Final Notes from This Run
==================================================================
1) Shard key choice used here:
   - member table: memberid
   - personvisit table: personid
   - routing rule: id % 3
   This keeps lookup/insert routing simple and deterministic.
2) Distribution observed in this notebook run:
   shard_0: members=1666 (33.32%), visits=16664 (33.32%)
   shard_1: members=1667 (33.34%), visits=16673 (33.34%)
   shard_2: members=1667 (33.34%), visits=16677 (33.34%)
3) Query behavior:
   - member-specific API calls are O(1): exactly one shard is touched.
   - dashboard/range/admin queries use scatter-gather across all 3 shards.
   - this is visible in Cell 7 and Cell 8 outputs.
4) Trade-offs I observed:
   - Adding more shards later is costly with hash-mod (rebalancing needed).
   - Even with equal member counts, visit traffic can still be uneven by shard.
   - Scatter-gather queries are slower than single-shard lookups.
   - Email login without a global secondary index needs fan-out querying.
5) Practical note:
   - With ~5000 members, sharding is not required for performance yet.
   - For this assignment, the main goal is proving correctness of routing and integrity checks.
==================================================================
```

Source: notebook output | lines shown: 25

Figure 24: Cell 9 trade-off analysis screenshot

6. Git Repository and Branch Strategy

Table 7: Git Branch Structure for Assignment 4

Branch	Role	Purpose	Key Files
`develop`	Integration	Receives merges from all A4 feature branches	*merged codebase on* ***`develop`***
`chore/a4-setup`	Feature	Assignment 4 folder baseline and docker setup	`docker-compose.yml`, `.env.example`
`feat/a4-shard-schema`	Feature	SQL DDL/DML for create, migrate, verify, indexes	`01_create_shards.sql` to `04_add_shard_indexes.sql`
`feat/a4-router-python`	Feature	Python shard routing and connection helpers	`connection.py`, `shard_router.py`
`test/a4-notebook-proof`	Feature	Notebook evidence for all sharding subtasks	`test_sharding.ipynb`, `shard_distribution.png`
`feat/a4-backend-routing`	Feature	JS router and shard-aware backend services	`shardRouter.js`, `member.service.js`, `personVisit.service.js`
`docs/a4-report`	Feature	Assignment 4 final report documentation	`GateGuard_Assignment4_Report.tex`

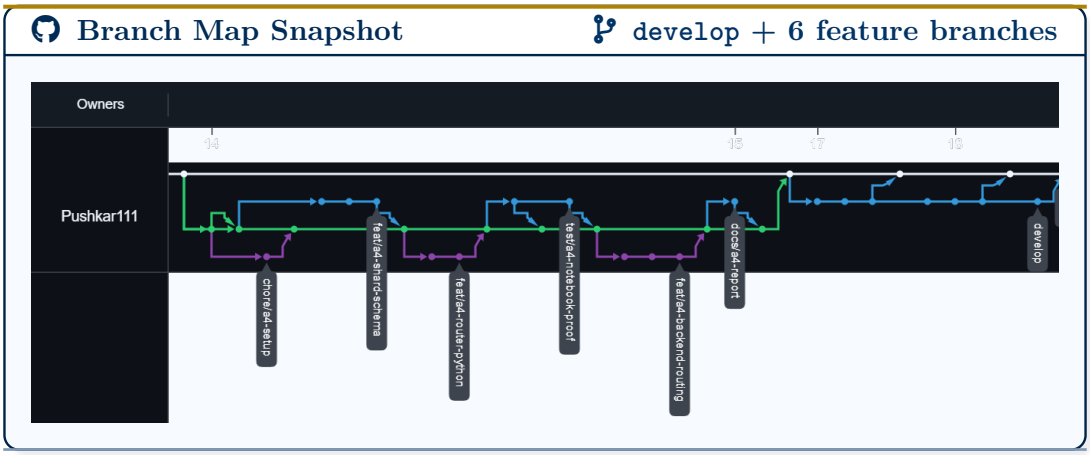


Figure 25: GitHub repository showing `develop` and six Assignment 4 feature branches, each scoped to a specific deliverable

7. Observations and Limitations

7.1 What Worked Well

The modulo hash function turned out to be the cleanest part of the system. We implemented it once in Python, once in JavaScript, and got deterministic behavior everywhere. The distribution came out almost perfectly balanced, which gave us confidence early.

The Assignment 3 concurrency fix carried over smoothly. Because routing logic is isolated, we only needed a small table-name substitution in the critical service path while keeping lock and transaction behavior intact. That separation made this upgrade much less risky than we expected.

The proof notebook became our strongest artifact. Every cell has a narrow purpose, every output is visible, and pass-fail outcomes are explicit. Anyone can rerun the notebook and compare outputs directly with the report.

7.2 Limitations

1. No distributed commit protocol exists across shards; cross-shard operations can show a consistency window.
2. Resharding to N+1 shards requires large-scale row movement and we do not implement online resharding in this assignment.
3. Email-based login still incurs scatter-gather overhead without a global secondary index.
4. Application-level router is a single point of failure in current architecture.
5. Dataset size does not require sharding yet; this is primarily a correctness and architecture exercise.

8. Conclusion

Final status: All Assignment 4 outcomes are complete and evidence-backed: deterministic shard key selection, data partitioning without overlap, single-shard and scatter-gather routing, backend integration, and explicit scalability trade-off analysis.

This Assignment 4 implementation covers the full sharding path across schema, migration, routing, and validation. We now have deterministic single-shard routing for key-based operations and working scatter-gather for global queries, both validated by notebook evidence and runtime checks.

The report also records trade-offs honestly. We documented where our design is strong, where it is intentionally simplified for assignment scope, and what would need to change for production scale. That balance was important for us because it proves we understand both implementation and system-level consequences.

Overall, the sharding architecture is complete for course requirements and is reproducible from code, SQL scripts, and notebook outputs in the repository.

9. Team Contributions

Contribution Atlas → Assignment 4

This section uses the same card-based contribution atlas style as Assignment 3: one lead card, four specialist cards, and a collaboration strip.

Core Build

Shard schema, hash routing, and backend integration

Validation

Notebook proofs, integrity checks, and routing verification

Delivery

Branch strategy, proof packaging, and release-ready report delivery

Pushkar Nikunjkumar Modi → Primary Implementation Lead

- Designed the full Assignment 4 sharding architecture: key strategy, routing model, and shard-level data ownership rules.
- Implemented and stabilized the SQL sharding pipeline (schema clone, migration, overlap verification, and index track).
- Owned Python and JavaScript routing integration, including service-level query-path updates and lock-safe behavior alignment.
- Drove notebook proof engineering (Cells 1–9), including deterministic routing checks, scatter-gather validation, and integrity assertions.
- Finalized technical narrative: trade-off analysis, cross-section consistency, and end-to-end reproducibility of submitted artifacts.

Krina Paghadar

- Refined Python router and Module A connection flow with function-level checks.
- Supported routing-integrity validation and shard-output cross-checks.
- Co-managed notebook evidence packaging and screenshot-to-section alignment.

Chintan Bhikhabhai Akhaja

- Supported SQL validation by checking shard counts, overlap results, and migration consistency.
- Assisted query-path review to align backend outputs with report tables and summaries.
- Co-managed final evidence formatting and section-to-output trace mapping.

Divisht

- Assisted backend smoke checks for routed visit-entry paths.
- Supported merge-time verification and issue triage.

Akshata Pawar

- Managed evidence curation and section-level asset alignment.
- Supported proofreading, format checks, and submission packaging.

Collaboration Note

Implementation ownership depth varies by subtask, while every member is credited for planning, iterative review, validation feedback, and final submission hardening.

A. Appendix A: Module_A Structure

```
assignment-4/Module_A/
|-- database/
|   |-- __init__.py
|   |-- connection.py
|   `-- shard_router.py
|-- schema/
|   |-- 01_create_shards.sql
|   |-- 02_migrate_data.sql
|   |-- 03_verify_migration.sql
|   `-- 04_add_shard_indexes.sql
|-- test_sharding.ipynb
|-- shard_distribution.png
`-- requirements.txt
```

Listing 18: Assignment 4 Module A tree

B. Appendix B: Module_B Backend Structure

```
assignment-4/Module_B/backend/src/
|-- utils/
|   `-- shardRouter.js
|-- services/
|   |-- personVisit.service.js
|   `-- member.service.js
`-- config/
    `-- db.js
```

Listing 19: Assignment 4 Module B backend src tree excerpt

The db.js connection setup is unchanged. Sharding currently targets Member and PersonVisit first, while smaller or lower-traffic tables continue through the existing base database flow.

C. Appendix C: .env.example Configuration

```
# Source database
LOCAL_DB_HOST=localhost
LOCAL_DB_PORT=5432
LOCAL_DB_NAME=gateguard
LOCAL_DB_USER=postgres
LOCAL_DB_PASSWORD=root

# Shard 0
SHARD_0_HOST=localhost
SHARD_0_PORT=5433
SHARD_0_DB=gateguard_shard_0
SHARD_0_USER=postgres
SHARD_0_PASSWORD=shard_root

# Shard 1
SHARD_1_HOST=localhost
```

```
SHARD_1_PORT=5434
SHARD_1_DB=gateguard_shard_1
SHARD_1_USER=postgres
SHARD_1_PASSWORD=shard_root

# Shard 2
SHARD_2_HOST=localhost
SHARD_2_PORT=5435
SHARD_2_DB=gateguard_shard_2
SHARD_2_USER=postgres
SHARD_2_PASSWORD=shard_root

NUM_SHARDS=3
```

Listing 20: Suggested environment variables for Assignment 4

D. Appendix D: References

Evidence Architecture Board

This reference section is structured as an academic evidence board: each source is linked to a concrete design consequence, where it is used, and why it raises technical rigor.

R1 | AWS: What Is Database Sharding?

Source Class: Industry systems guide **Reliability Tier:** A
Provenance: AWS architecture knowledge base
Contribution: Established the scale-out framing and shard-isolation motivation for our architecture narrative.

Design Impact: Strengthened deterministic hash routing as the default low-latency path for member-scoped operations.

Trace Location: Sections 2.2, 2.4, 5.1
URL: https://aws.amazon.com/what-is/database-sharding/

R2 | PingCAP: Database Sharding Defined

Source Class: Industry design primer **Reliability Tier:** A-
Provenance: Multi-region engineering publication
Contribution: Informed explicit discussion of fan-out overhead, rebalance cost, and operational trade-offs.

Design Impact: Anchored the trade-off matrix used in Section 5 for scalability and complexity analysis.

Trace Location: Section 5
URL: https://www.pingcap.com/blog/database-sharding-defined/

R3 | PostgreSQL CREATE TABLE

Source Class: Official DBMS documentation **Reliability Tier:** A+
Provenance: PostgreSQL core documentation
Contribution: Validated schema cloning strategy for shard-table bootstrap using native SQL semantics.

Design Impact: Justified `LIKE ... INCLUDING ALL` to preserve defaults and structural compatibility across shards.

Trace Location: Section 3.3, Script 01
URL: https://www.postgresql.org/docs/current/sql-createtable.html

R4 | PostgreSQL FOR UPDATE

Source Class: Official DBMS concurrency documentation **Reliability Tier:** A+
Provenance: PostgreSQL transaction semantics reference
Contribution: Grounded lock behavior required for safe concurrent entry flows under sharded writes.

Design Impact: Confirmed row-level locking strategy that preserves ACID correctness under contention.

Trace Location: Section 4.3.2
URL: https://www.postgresql.org/docs/current/sql-select.html#SQL-FOR-UPDATE-SHARE